# Software Design Document

# Financial Investment Monitoring System - Software Specification Document

## Purpose

The purpose of this document is to provide a detailed software specification for the development of a financial investment monitoring system. This system is designed to monitor clients' investments, ensuring constant availability, privacy, and data integrity. The document outlines the architectural design, major design decisions, and testing strategies for successful system development and deployment.

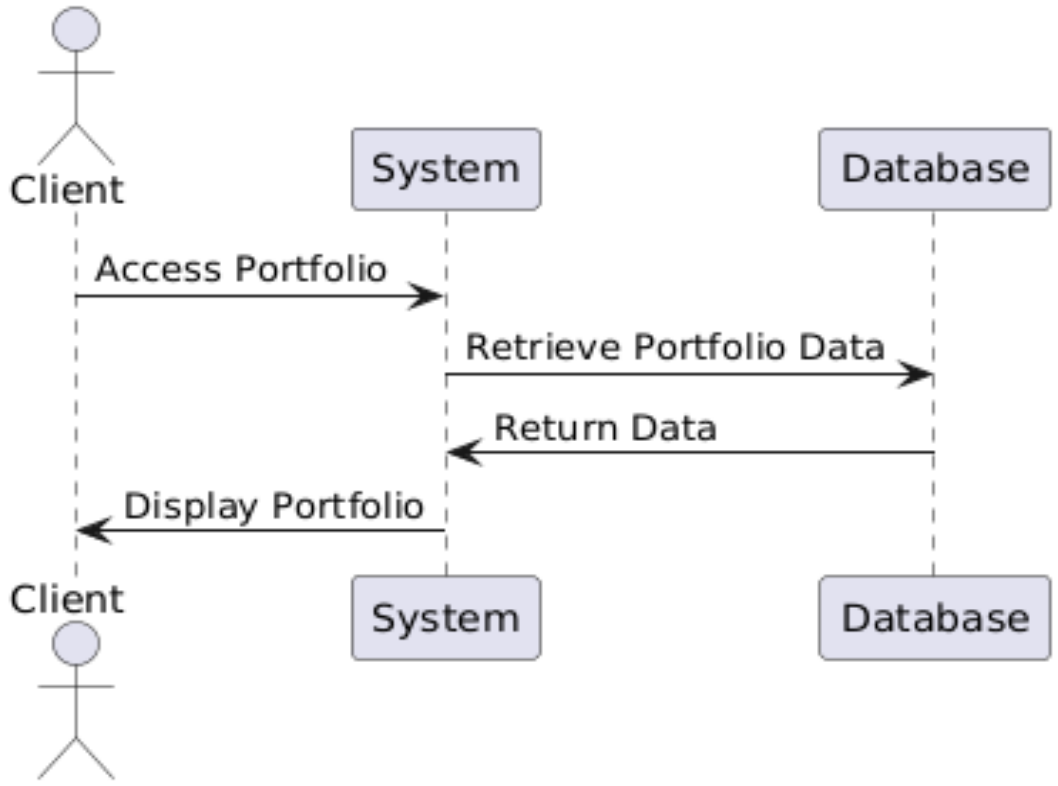
## Use Cases and Sequence Diagrams

### Use Case UC01: Monitor Investment Portfolio

- **Identifier:** UC01

- **Description:** Enables clients to view and monitor their investment portfolios in real-time.

```plantuml
```



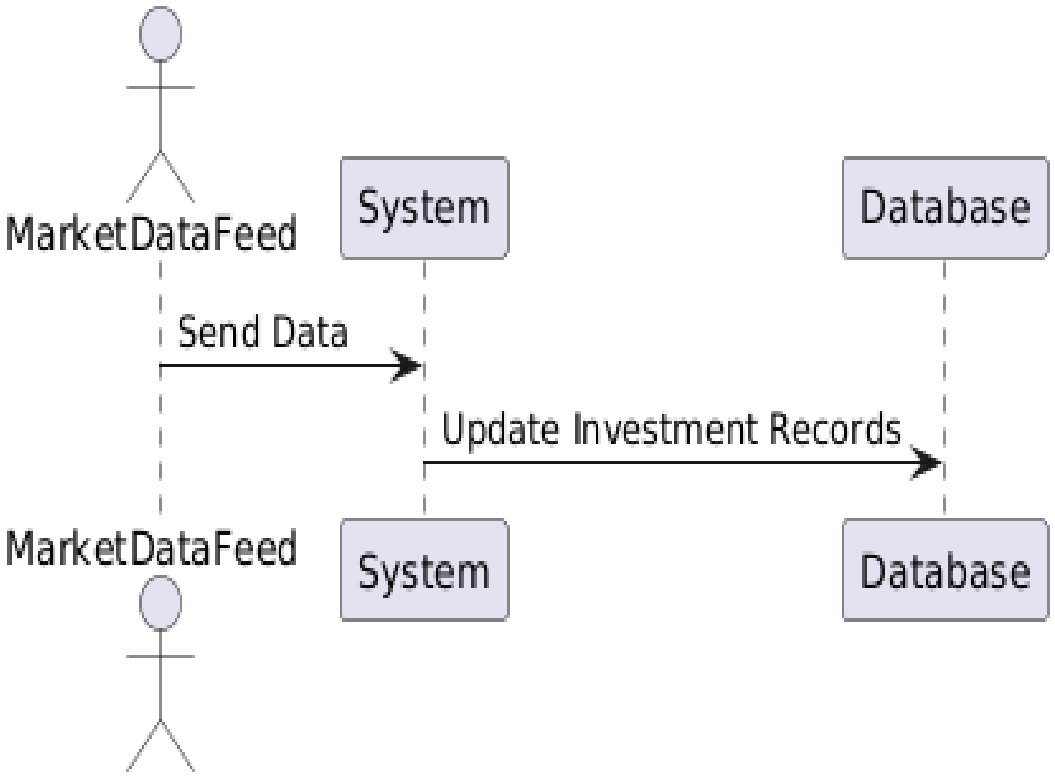

```

**Use Case UC02: Update Investment Data**

- **Identifier:** UC02

- **Description:** Regularly updates investment data leveraging real-time market data feeds.

```plantuml




```

## Major Design Decisions

### Design Choices and Modularization

- **Choice of Cloud Hosting:** The system will be deployed on a cloud platform to ensure scalability and availability.

- **Microservices Architecture:** The system will be separated into distinct services, each responsible for a specific functionality.

### Cohesion and Coupling

- **High Cohesion:** Each service module will have a clear and singular purpose (e.g., Portfolio Management Service).

- **Low Coupling:** Services will communicate via well-defined APIs to minimize dependencies.

### Non-Functional Requirements
```

- **Security:** Strong encryption and authentication mechanisms will be utilized to protect client data.

- **Reliability:** High uptime will be ensured using redundant resources and failover strategies.

## Architecture

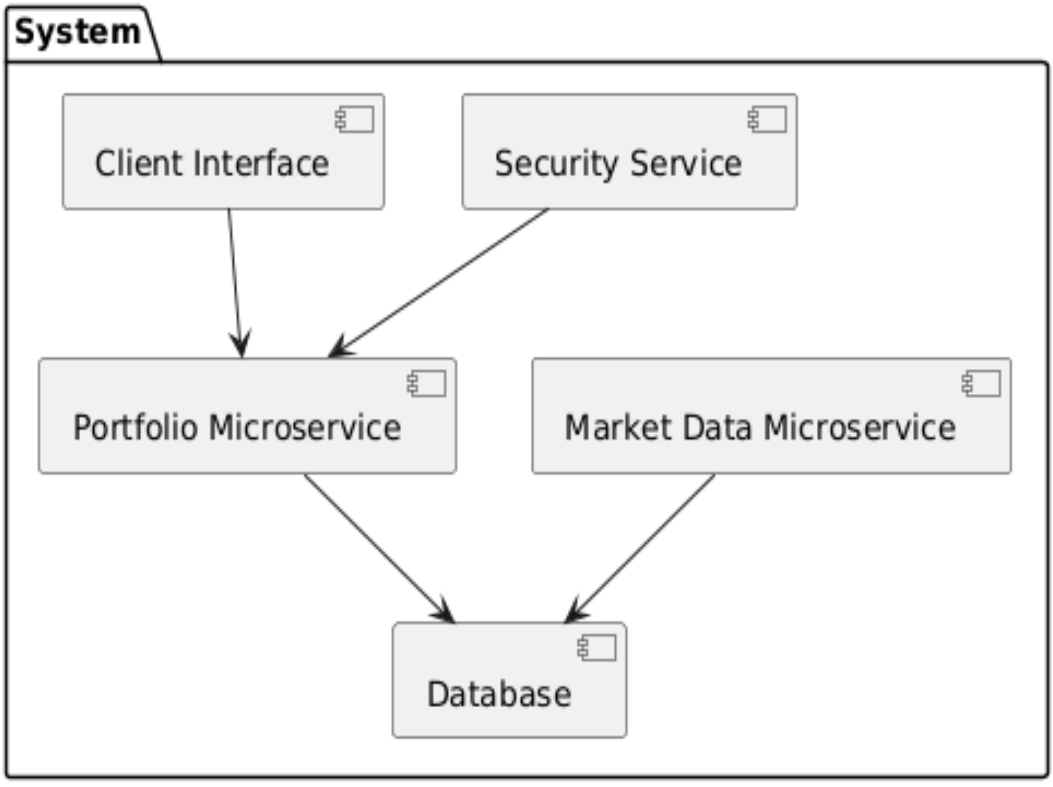
### Architectural Patterns and Rationale

- **Microservices Architecture:** This pattern allows for independent deployment, scaling, and management of system components, providing flexibility and resilience. Each microservice can be developed using the most appropriate technology stack.

- **Centralized Logging and Monitoring:** A centralized system to log and monitor activities will help in maintaining the system's reliability and quickly identifying issues.

### System Decomposition and Components

```plantuml
```




```

- **Client Interface:** Provides the user interface for clients to interact with the system.
```

- **Portfolio Microservice:** Manages investment portfolio data and interactions.

- **Market Data Microservice:** Handles market data feeds and updates investment records.

- **Security Service:** Enforces authentication and authorization policies.

- **Database:** Stores client portfolio and investment data.

**Exposed Interfaces and Operations**

- **Client Interface:**

- `login(username, password)`: Authenticates a client.

- `viewPortfolio(clientID)`: Returns the current state of the client's portfolio.

- **Portfolio Microservice:**

- `getPortfolioData(clientID)`: Retrieves portfolio data.

- `updatePortfolioData(data)`: Updates investment records.

- **Market Data Microservice:**

- `receiveMarketData(data)`: Processes incoming market data.

- **Security Service:**

- `authenticateRequest(token)`: Validates client requests.

## Use of Design Patterns

- **Strategy Pattern:** Used within the Market Data Microservice to handle different data formats from various sources.

- **Observer Pattern:** Allows the system to react to market data changes and update portfolios accordingly.

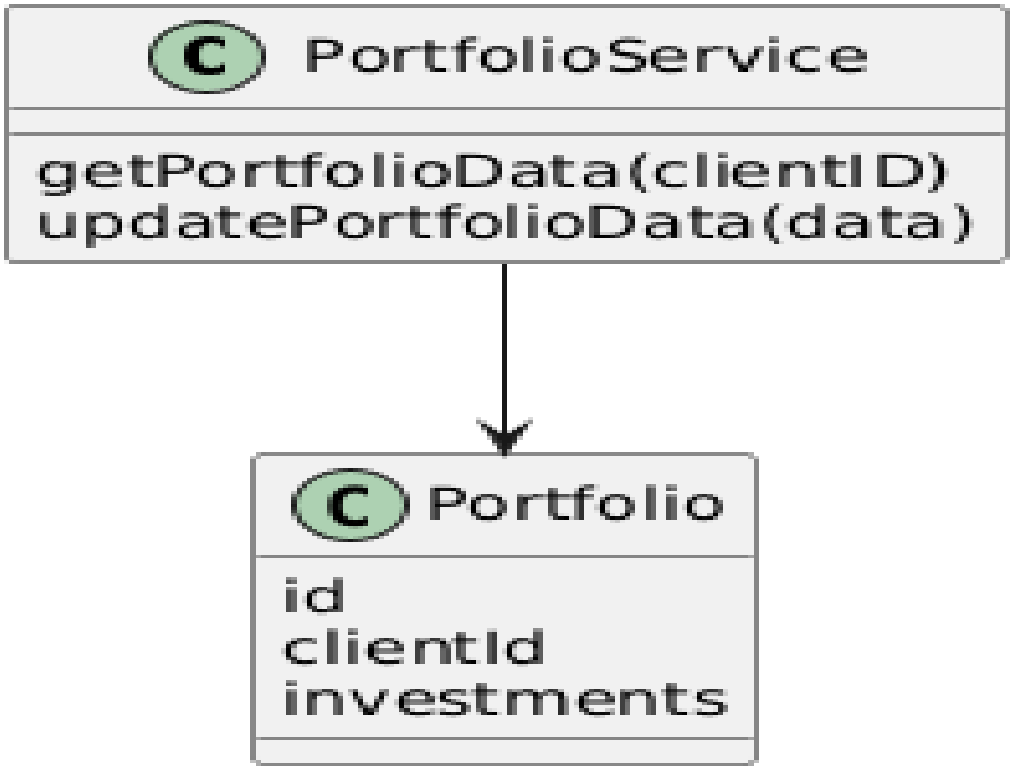
- **Facade Pattern:** In the Client Interface to simplify interactions by providing a unified interface.

## Detailed Class Diagrams

Each microservice will have a corresponding UML class diagram detailing its internal structure and relationships.

**Portfolio Microservice**

```plantuml
```




```

## Test Driven Development

**Test Case TC01**

- **Test ID:** TC01

- **Category:** Portfolio Data Retrieval

- **Initial Condition:** Client is authenticated, and portfolio data exists.

- **Procedure:**
```

1. Execute `getPortfolioData(clientID)`.

- **Expected Outcome:** Portfolio data is returned accurately.

**Test Case TC02**

- **Test ID:** TC02

- **Category:** Market Data Update

- **Initial Condition:** Market data changes.

- **Procedure:**

1. Simulate a market data feed.

2. Execute `receiveMarketData(data)`.

- **Expected Outcome:** Investment records in the database are updated.

This document provides an initial framework and guide for the development process, which may be iteratively refined as the system design progresses.